Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Никонова А.Э.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 16.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить. Задаётся радиус окружности. Необходимо с помощью метода Монте-Карло рассчитать её площадь.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create - создаёт новый поток выполнения;
- pthread_join - блокирует главный поток до завершения указанного рабочего потока; вызывается для каждого успешно созданного потока (счётчик created);
- pthread_mutex_init - инициализирует мьютекс с атрибутами по умолчанию; вызывается один раз в начале run_parallel перед созданием потоков.
- pthread_mutex_lock - захватывает мьютекс, блокируя поток при занятости; используется при выдаче очередной порции работы (next_chunk) и при добавлении локального результата в общий (inside_total);
- pthread_mutex_unlock - освобождает мьютекс, будя один из ожидающих потоков; парная к lock;
- pthread_mutex_destroy - уничтожает мьютекс после завершения всех потоков; вызывается в конце run_parallel, в том числе на пути обработки ошибок.
- clock_gettime - возвращает текущее время;
- calloc - выделяет и обнуляет память под массив идентификаторов потоков pthread_t *tid = calloc(nthreads, sizeof *tid); возвращает NULL при нехватке памяти, что проверяется перед созданием потоков.
- free - освобождает память, выделенную calloc под массив tid, в конце run_parallel.
- printf - форматированный вывод в stdout;
- fprintf - форматированный вывод в stderr: сообщения об ошибках;
- snprintf - безопасное форматирование в буфер фиксированного размера; используется при сборке строк таблицы;
- puts - вывод строки с добавлением \n; используется в print_row для завершения строки таблицы.
- putchar - вывод одного символа; используется в print_sep для рисования горизонтальных линий рамки таблицы (+, -).
- strcmp - сравнение строк; используется в main для распознавания ключей запуска.
- atof - преобразование строки в double; используется для разбора --radius.
- strtoull - преобразование строки в unsigned long long; используется для разбора --points (число точек N).
- atoi - преобразование строки в int; используется для разбора --threads, --max-threads и чисел после --sweep.

- strerror - возвращает текстовое описание кода ошибки; используется при печати сообщения pthread_create[%d] failed: %s, чтобы вместо числового кода было читаемое описание.
- fabs - модуль числа с плавающей точкой; используется для расчёта относительной ошибки error_seq и error_par при сравнении с точным значением πR^2

main читает ключи --radius, --points, --threads, --max-threads, --sweep. Число потоков ограничивается: nthreads = min(--threads, --max-threads). Ключ --sweep собирает список значений p для построения таблицы.

Последовательная версия. run_sequential(N, seed) инициализирует генератор xs64 начальным seed, вызывает do_chunk(N) - один цикл на N итераций, каждая генерирует пару (x, y) и проверяет попадание. Возвращает K. Время меряется через clock_gettime до и после.

Параллельная версия. run_parallel(N, p, seed, &K):Инициализирует общий контекст mc_ctx: N, размер порции chunk = 65536, счётчик next_chunk = 0, аккумулятор inside_total = 0, мьютекс. Создаёт p потоков через pthread_create; тело потока - worker. Каждый worker: инициализирует свой генератор xs64 от уникального seed (полученного из общего seed_base через xs64), чтобы потоки не генерировали одинаковые последовательности; в цикле под pthread_mutex_lock берёт следующую порцию [start, start+take) из next_chunk, освобождает мьютекс и обрабатывает её вне критической секции (do_chunk), копя локальный local_in; когда next_chunk >= N — выходит из цикла, под мьютексом добавляет local_in в inside_total. Главный поток pthread_join-ит всех рабочих, уничтожает мьютекс, освобождает tid, возвращает inside_total. Размер порции 65536 выбран так, чтобы захватов мьютекса было мало, а дисбаланс последней порции - пренебрежимо мал относительно 10^8 итераций.

Подсчёт метрик. S(p) = T_seq / T_par(p) - во сколько раз параллельная версия быстрее последовательной. E(p) = S(p) / p - доля полезной загрузки каждого потока (E = 1 - идеал, E → 0 - потоки простаивают). В --sweep T_seq измеряется один раз, и все S(p) считаются от одного эталона.

Код программы

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <time.h>
#include <pthread.h>
#include <math.h>

typedef struct {
    double   radius;
    uint64_t total_points;
    uint64_t chunk;
    uint64_t next_chunk;
    uint64_t inside_total;
    uint64_t seed_base;
    pthread_mutex_t mtx;
} mc_ctx;
```

```c
static inline uint64_t xs64(uint64_t *s) {
    uint64_t x = *s;
    x ^= x >> 12; x ^= x << 25; x ^= x >> 27;
    *s = x;
    return x * 0x2545F4914F6CDD1DULL;
}

static inline double u01(uint64_t *s) {
    return (xs64(s) >> 11) * (1.0 / 9007199254740992.0);
}

static uint64_t do_chunk(uint64_t n, double r2, uint64_t *rng) {
    uint64_t in = 0;
    for (uint64_t i = 0; i < n; ++i) {
        double x = 2.0 * u01(rng) - 1.0;
        double y = 2.0 * u01(rng) - 1.0;
        if (x*x + y*y <= 1.0) in++;
    }
    (void)r2;
    return in;
}

static uint64_t run_sequential(uint64_t N, uint64_t seed) {
    uint64_t rng = seed | 1;
    return do_chunk(N, 1.0, &rng);
}

static void *worker(void *arg) {
    mc_ctx *c = (mc_ctx *)arg;
    uint64_t rng = xs64(&c->seed_base) | 1;
    uint64_t local_in = 0;

    for (;;) {
        pthread_mutex_lock(&c->mtx);
        uint64_t start = c->next_chunk;
        uint64_t take  = c->chunk;
        if (start >= c->total_points) { pthread_mutex_unlock(&c->mtx); break; }
        if (start + take > c->total_points) take = c->total_points - start;
        c->next_chunk += take;
        pthread_mutex_unlock(&c->mtx);

        local_in += do_chunk(take, 1.0, &rng);
    }

    pthread_mutex_lock(&c->mtx);
    c->inside_total += local_in;
    pthread_mutex_unlock(&c->mtx);
    return NULL;
}

static double run_parallel(uint64_t N, int nthreads, uint64_t seed, uint64_t *out_in) {
    mc_ctx c = {0};
    c.total_points = N;
    c.chunk        = 1 << 16;
```

```
    c.next_chunk   = 0;
    c.inside_total = 0;
    c.seed_base    = seed | 1;

    if (pthread_mutex_init(&c.mtx, NULL) != 0) {
        fprintf(stderr, "pthread_mutex_init failed\n");
        *out_in = 0;
        return -1.0;
    }

    pthread_t *tid = calloc(nthreads, sizeof *tid);
    if (!tid) {
        fprintf(stderr, "calloc failed for %d threads\n", nthreads);
        pthread_mutex_destroy(&c.mtx);
        *out_in = 0;
        return -1.0;
    }

    int created = 0;
    for (int i = 0; i < nthreads; ++i) {
        int rc = pthread_create(&tid[i], NULL, worker, &c);
        if (rc != 0) {
            fprintf(stderr, "pthread_create[%d] failed: %s\n", i, strerror(rc));
            break;
        }
        created++;
    }
    for (int i = 0; i < created; ++i)
        pthread_join(tid[i], NULL);

    pthread_mutex_destroy(&c.mtx);
    free(tid);
    *out_in = c.inside_total;
    return 0.0;
}

static double now_ms(void) {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e3 + ts.tv_nsec / 1e6;
}
static void print_sep(int w[], int n) {
    for (int i = 0; i < n; ++i) { putchar('+'); for (int j = 0; j < w[i]; ++j) putchar('-
'); }
    puts("+");
}

static void print_row(const char **cells, int w[], int n) {
    for (int i = 0; i < n; ++i) printf("| %-*s ", w[i]-1, cells[i]);
    puts("|");
}

static void run_sweep(uint64_t N, double R, const int *ps, int np, double T_seq) {
    int w[4] = {15, 18, 12, 14};
    const char *hdr[4] = {"Threads", "Time (ms)", "Speedup", "Efficiency"};
```

```
    print_sep(w, 4);
    print_row(hdr, w, 4);
    print_sep(w, 4);

    char buf[4][32];
    const char *cells[4] = {buf[0], buf[1], buf[2], buf[3]};

    snprintf(buf[0], 32, "%d (seq)", 1);
    snprintf(buf[1], 32, "%.2f", T_seq);
    snprintf(buf[2], 32, "%.2f", 1.0);
    snprintf(buf[3], 32, "%.2f", 1.0);
    print_row(cells, w, 4);

    for (int i = 0; i < np; ++i) {
        int p = ps[i];
        uint64_t inside = 0;
        double t0 = now_ms();
        run_parallel(N, p, 0xDEADBEEFCAFEBABEULL, &inside);
        double t = now_ms() - t0;

        double S = T_seq / t;
        double E = S / p;

        snprintf(buf[0], 32, "%d", p);
        snprintf(buf[1], 32, "%.2f", t);
        snprintf(buf[2], 32, "%.2f", S);
        snprintf(buf[3], 32, "%.3f", E);
        print_row(cells, w, 4);
    }
    print_sep(w, 4);
}
int main(int argc, char **argv) {
    double   R           = 1.0;
    uint64_t N           = 100000000ULL;
    int      nthreads    = 4;
    int      max_threads = 1024;

    int do_sweep = 1;                                          // ← дефолт
    int sweep_ps[16] = {1, 2, 3, 4, 16, 128, 1024};            // ← дефолт
    int sweep_np = 7;                                          // ← дефолт
    int verbose = 0;                                           // ← если делали --verbose

    for (int i = 1; i < argc; ++i) {
        if      (!strcmp(argv[i], "--radius")      && i+1<argc) R = atof(argv[++i]);
        else if (!strcmp(argv[i], "--points")      && i+1<argc) N = strtoull(argv[++i], 0,
10);
        else if (!strcmp(argv[i], "--threads")     && i+1<argc) nthreads =
atoi(argv[++i]);
        else if (!strcmp(argv[i], "--max-threads") && i+1<argc) max_threads =
atoi(argv[++i]);
        else if (!strcmp(argv[i], "--verbose") || !strcmp(argv[i], "-v")) verbose = 1;
        else if (!strcmp(argv[i], "--no-sweep")) do_sweep = 0;
        else if (!strcmp(argv[i], "--sweep")) {
```

```
            do_sweep = 1;
            sweep_np = 0;                                   // ← сброс
            while (i+1 < argc && argv[i+1][0] != '-') {
                if (sweep_np < 16) sweep_ps[sweep_np++] = atoi(argv[++i]);
                else ++i;
            }
        }
    }

    if (nthreads > max_threads) nthreads = max_threads;

    double t0 = now_ms();
    uint64_t in_seq = run_sequential(N, 0xDEADBEEFCAFEBABEULL);
    double t_seq = now_ms() - t0;
    double S_seq = 4.0 * R * R * (double)in_seq / (double)N;

    printf("R=%.4f  N=%llu  threads=%d\n",
           R, (unsigned long long)N, nthreads);
    printf("[seq] time = %.2f ms, S = %.6f (pi=%.6f)\n",
           t_seq, S_seq, S_seq / (R*R));

    uint64_t in_par = 0;
    t0 = now_ms();
    run_parallel(N, nthreads, 0xDEADBEEFCAFEBABEULL, &in_par);
    double t_par = now_ms() - t0;
    double S_par = 4.0 * R * R * (double)in_par / (double)N;

    printf("[par] time = %.2f ms, S = %.6f (pi=%.6f)\n",
           t_par, S_par, S_par / (R*R));
    printf("true S = %.6f, error_seq = %.4f%%, error_par = %.4f%%\n",
           3.14159265358979*R*R,
           100.0*fabs(S_seq - 3.14159265358979*R*R)/(3.14159265358979*R*R),
           100.0*fabs(S_par - 3.14159265358979*R*R)/(3.14159265358979*R*R));

    if (do_sweep) {
        printf("\n");
        run_sweep(N, R, sweep_ps, sweep_np, t_seq);
    }

    return 0;
}
```

Протокол работы программы

```
R=1.0000  N=100000000  threads=4
[seq] time = 772.35 ms, S = 3.141441 (pi=3.141441)
[par] time = 232.14 ms, S = 3.141421 (pi=3.141421)
true S = 3.141593, error_seq = 0.0048%, error_par = 0.0055%

+--------------+------------------+-----------+--------------+
| Threads      | Time (ms)        | Speedup   | Efficiency   |
+--------------+------------------+-----------+--------------+
| 1 (seq)      | 772.35           | 1.00      | 1.00         |
| 1            | 764.53           | 1.01      | 1.010        |
| 2            | 375.96           | 2.05      | 1.027        |
| 3            | 284.78           | 2.71      | 0.904        |
| 4            | 244.97           | 3.15      | 0.788        |
| 16           | 216.14           | 3.57      | 0.223        |
| 128          | 222.59           | 3.47      | 0.027        |
| 1024         | 288.95           | 2.67      | 0.003        |
+--------------+------------------+-----------+--------------+
```

Вывод

Не знала о существовании метода Монте-Карло. Очень долго не могла разобраться в построении таблицы для всех экспериментов, как именно рассчитать ускорение и эффективность, как управлять несколькими потоками. Лабораторная работа показалась очень сложной и времязатратной. Однако я овладела многопоточностью благодаря ей.